

Exercise 05: Reducing & Saturating

With digital control schemes, control input decisions are made based on error terms. Take our specific case of "Segway" balance control. If the platform is pitching forward more than desired, we want to drive the motors harder forward. If this pitch error is small we want to make a slight adjustment (we want good granularity). If this pitch error is large we want to make a large adjustment. This implies we need a lot of bits to store the error, so we can have both fine granularity and large range.

However, in control systems there gets to be a point where we don't care how large the error is, we just need to know it is large and the system should do as much as it can to correct it. If the "Segway" platform is pitched forward way more than we want it to, we no longer care exactly how much, at that point we know we want to drive the motors forward really hard.

So a wide error term could be reduced and saturated to a smaller number of bits to make down stream calculations narrower (fewer bits wide) and thus use fewer gates.

This is something we will do in the balance controller math. OK…now that I have setup the motivation for the problem. Here is the problem:

You are going to write a **dataflow Verilog** module with the following interface:

Exercise 05: Reducing & Saturating

Signal:	Dir:	Description:
unsigned_err[15:0]	in	16-bit **unsigned** error term to be reduced/saturated to 10-bits
unsigned_err_sat[9:0]	out	10-bit saturated version of unsigned_err[15:0]. *How would unsigned saturation to a 10-bit value work?*
signed_err[15:0]	in	16-bit **signed** number to be reduced/saturated to 10-bits.
signed_err_sat[9:0]	out	10-bit saturated version of signed_err[15:0]. If result is positive and greater than 0x1FF then it saturates to 0x1FF. If result is more negative than 0x200 then it saturates to 0x200
signed_D_diff[9:0]	in	10-bit signed version of a difference term used for derivative calculation
signed_D_diff_sat[6:0]	out	7-bit signed saturated version of signed_D_diff[9:0]

The name of the DUT should be **saturate.sv**

The name of the testbench should be **saturate_tb.sv** (should be self-checking)

Your testbench should apply several values to each input and observe the saturation of the outputs. You should choose several values that would cause the signed value to saturate in various ways.

Hints on Saturating

- Your saturation implementation will be used in your final project where the overall circuit area is a grading metric. You want to choose a design that will give you lower area when synthesized.

- 8-bit to 5bit example:

  01010110 ← this is positive because MSB is 0
  Greatest positive number in 5-bits = **01111**
  01010110 > **01111**? Yes. How do we know? Because it is positive and it has bit(s) in the [6:4] range set. So we saturate to **01111**

  11010110 ← this is negative because MSB is 1
  Greatest negative number 5-bits = **10000**
  11010110 < **10000**? Yes. How do we know? Because it is negative and it has bit(s) in the [6:4] range clear. So we saturate to **10000**

  11110110 ← this is negative because MSB is 1
  11110110 < **10000**? No. How do we know? Because it is negative but all the bits in the [6:4] are also set. So we simply keep bits [4:0] as our 5-bit result.

- Is there an efficient way to checks if a range of bits are 0 or 1? Perhaps an operator?